Cryptography and VRFs

The single idea Algorand is built on, and the maths that makes fake identities worthless. Everything here becomes code you can read in `engine/vrf/`.

CONTENTS

1 **What Phase 2 is for** — the one idea everything points at

2 **Hashing, properly** — three resistances, not one

3 **The birthday bound** — why collisions cost half the bits

4 **Merkle trees** — proving membership without the data

5 **One-way functions** — and a gentle look at elliptic curves

6 **Ed25519** — and why determinism matters here

7 **Verifiable Random Functions** — the heart of the whole project

8 **Sortition** — how Algorand runs a secret lottery

9 **The binomial and sub-users** — turning stake into votes

10 **Sybil resistance** — the identity that kills fake accounts

11 **Your engine's shortcut** — what it does, and where it falls short

12 **Real VRFs** — RFC 9381, and when you would need it

13 **Glossary** — every new term in one place

14 **Build it yourself** — five exercises, all runnable today

15 **Checkpoint** — questions, then answers

BEFORE YOU START

How to read this document

Phase 2 of your eight-phase plan. Budget about ten hours — it is the second-longest phase, and the only one whose subject is a single idea.

This document assumes Phase 1. If "hash", "signature", "Sybil" or "consensus" are not yet comfortable words, go back — nothing here will land otherwise.

Sections 1 to 6 are the run-up. They deepen what Phase 1 introduced. Read them properly, but do not linger: they exist to make section 7 possible.

Section 7 is the point of the entire phase. If you read one section twice, read that one. Everything in your project's `engine/vrf/` package is section 7 turned into Go.

Section 14 has five exercises, and all five run on the Python you already have. No installs. They were each executed while this document was written, and the outputs shown are the real ones.

WHAT YOU WILL BE ABLE TO DO AFTERWARDS

State the three properties of a VRF, and say exactly what a VRF gives you that a signature does not.

Explain why splitting a stake across a hundred fake accounts wins an attacker nothing — using the actual identity, not a hand-wave.

Read `engine/vrf/sortition.go` line by line and say what every line is for, including the binomial walk.

Say precisely why the Ed25519 shortcut in that file is not a real VRF, and why it is still the right call for a permissioned Fabric network.

A NOTE ON THE MATHS

There is more arithmetic here than in Phase 1, and none of it is optional — the binomial is the Sybil defence, not decoration.

But you need no proofs and no advanced background. If you can follow "probability of k heads in n tosses", you can follow all of it. Elliptic curves are described in section 5 without a single field equation, because you do not need one to use them.

SECTION 1

What this phase is for

Phase 1 ended with a problem left open.

You know that consensus needs a committee, that the committee must be chosen fairly, and that it must be weighted by stake so fake identities gain nothing. But how do you actually run that lottery?

> *Whoever runs the draw can rig the draw. And if the winners are announced in advance, an attacker knows exactly whom to silence.*

Algorand's answer is a cryptographic primitive most people have never heard of: the **Verifiable Random Function**. It lets every participant run the lottery privately, on their own machine, and later prove the result — with no organiser, no announcement, and no way to cheat.

That is section 7. Sections 2 through 6 build the vocabulary it needs.

SECTION	WHAT IT ADDS	WHY SECTION 7 NEEDS IT
2 – 3	Hashing, in real detail	A VRF's output must be indistinguishable from random
4	Merkle trees	Not needed for VRFs — but Fabric uses them, and Phase 7 will assume you know
5	One-way functions, curves	A VRF is built on the same hardness as a key pair
6	Ed25519 and determinism	Your engine's VRF **is** an Ed25519 signature. The determinism is the trick

KEEP ONE QUESTION IN YOUR HEAD THROUGHOUT

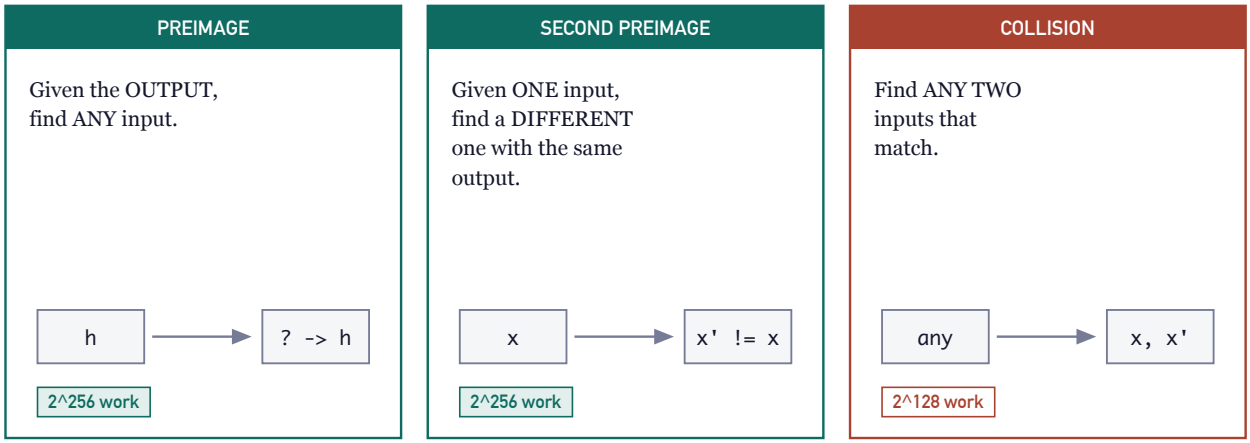
How do you produce a random number that you cannot choose, that nobody can predict, and that you can nonetheless prove is the right one?

Every section from here to section 7 is a piece of that answer. When something feels like a detour, ask how it serves that question.

SECTION 2

Hashing, properly this time

Phase 1 said a hash is one-way. That was true but incomplete: "one-way" is actually three separate promises, and they are not equally strong.



COLLISION IS THE WEAK ONE — AND IT IS THE ONE A BLOCKCHAIN LEANS ON.
Half the bit-length, because of the birthday bound. See Figure 2.

Figure 1 — Three different promises, each harder to break than it sounds.

Preimage resistance	Given an output h, you cannot find any input that hashes to it. This is what stops someone reversing a hash back into the data.
Second-preimage resistance	Given a specific input x, you cannot find a different x' with the same hash. This is what stops someone swapping one transaction for another inside a block while keeping the block's hash intact.
Collision resistance	You cannot find **any** two inputs that hash to the same value. Note the freedom: the attacker gets to choose *both*. That extra freedom is exactly why this is the weakest of the three.

The distinction matters for your project. Every block stores its predecessor's hash, so the chain's immutability rests on **second-preimage resistance**. But a system where an attacker can craft two colliding blocks from scratch — one benign, one malicious — is broken by **collision resistance** alone. That is the one that gives way first, and section 3 shows by how much.

SECTION 3

The birthday bound

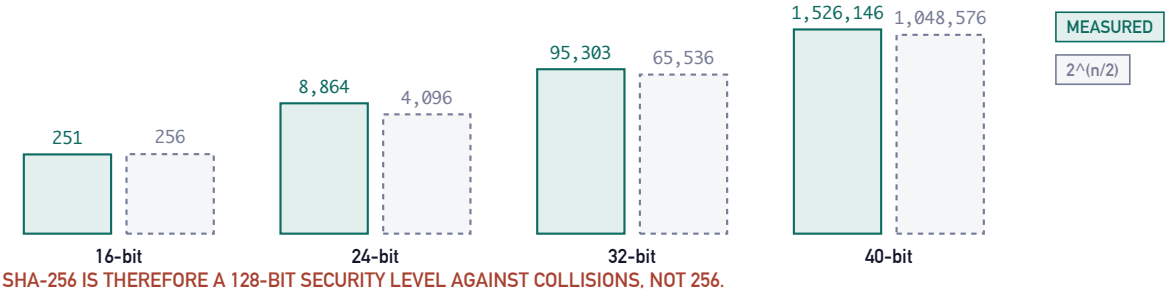
Here is a fact that surprises almost everyone the first time.

In a room of 23 people, there is a better-than-even chance that two share a birthday. Not that someone shares *yours* — that would need about 253 people. Just that **some** pair matches.

The reason is that you are not making 23 comparisons, you are making every possible pair: 23 × 22 / 2 = 253 of them. Pairs grow with the *square* of the group.

WHY 'HALF THE BITS'

With n possible outputs, you might expect to try n times for a match. You do not.
Every NEW item is compared against every one already seen, so pairs grow quadratically.



SHA-256 IS THEREFORE A 128-BIT SECURITY LEVEL AGAINST COLLISIONS, NOT 256.

Figure 2 — Measured, not asserted: collisions appear after roughly the square root of the space.

Applied to hashes: with an n-bit output there are 2^n possible values, but a collision shows up after roughly 2^(n/2) attempts, not 2^n.

WHAT THIS MEANS IN PRACTICE

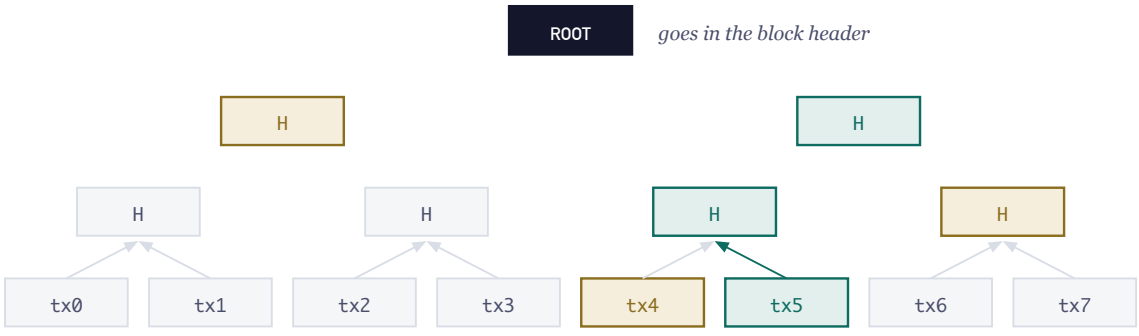
SHA-256 gives **256-bit** preimage resistance but only **128-bit** collision resistance. 128 bits is still far beyond reach, so this is a design fact, not a weakness — but it is why hash outputs are twice as long as the security level they are chosen for.

It is also why truncating a hash to "save space" is so dangerous. Cut SHA-256 down to 64 bits and you have 32-bit collision resistance: about four billion tries, which is minutes on a laptop. The numbers in Figure 2 were produced exactly that way.

SECTION 4

Merkle trees

A way to commit to a whole set of items with one hash, and then prove any single item was in that set without revealing or transmitting the rest.



TO PROVE tx5 IS IN THIS BLOCK YOU SEND:

tx5 itself, plus the three gold hashes. Not the other seven transactions.

The verifier re-hashes upward and checks it lands on the root they already trust.

8 LEAVES -> 3 HASHES 1024 LEAVES -> 10 HASHES GROWTH IS log2(n)

This is how a light client verifies a payment without downloading the chain.

Figure 3 — Prove one transaction is in a block of a thousand, using ten hashes.

Hash the items in pairs, then hash those hashes in pairs, and keep going until one value remains: the **root**. Change any leaf and the root changes, by the avalanche property.

The useful part is the **inclusion proof**. To convince someone that `tx5` is in the set, you send `tx5` plus one sibling hash per level. They recompute upward and compare against the root they already trust.

WHERE THIS SHOWS UP IN YOUR PROJECT

Fabric puts a Merkle-style hash over a block's transactions in the block header, so a peer can verify a block's contents are intact without re-reading every transaction.

It is **not** part of BA*, and your engine does not implement one. It is here because Phase 7 will show you Fabric block structure and assume you recognise it.

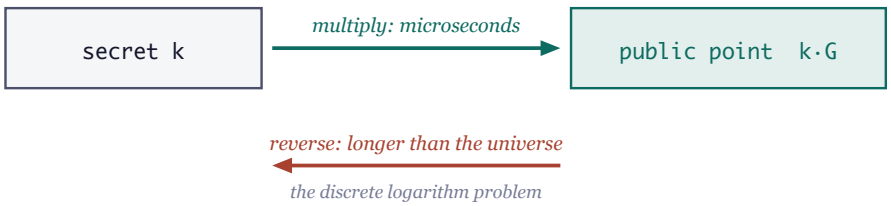
The wider idea — **commit to a lot with a little, then prove membership cheaply** — is worth carrying with you. It is the same instinct as a VRF proof: a small token that lets someone else check a claim they could not compute themselves.

SECTION 5

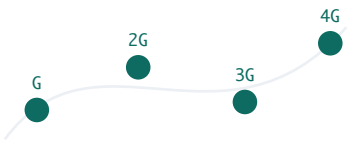
One-way functions and elliptic curves

Phase 1 said you can compute a public key from a private key but never the reverse. This section says what that actually is — without the algebra.

THE SHAPE OF THE PROBLEM



WHAT 'kG' MEANS, WITHOUT THE ALGEBRA



A curve is just a set of points with a rule for "adding" two of them.
Start at a fixed point G and add it to itself k times: that is k·G.
Doing it is fast even for astronomically large k (double-and-add).
Recovering k from the landing point is the hard part. That gap IS the key pair:
k is your private key, k·G is your public key.

Figure 4 — Easy one way, infeasible the other. Every public-key system is built on such a gap.

What you genuinely need to know

1. An elliptic curve is a set of points, plus a rule for "adding" two points to get a third. The rule is geometric and you do not need it.
2. Fix one point `G`, the **generator**. Adding it to itself `k` times gives a point written `k·G`.
3. Computing `k·G` is fast even when `k` has 256 bits, because you can double repeatedly instead of adding one at a time.

4. Recovering `k` from `k·G` is the **discrete logarithm problem**, and nobody knows how to do it in reasonable time.

That asymmetry **is** the key pair. Your private key is the number `k`; your public key is the point `k·G`. Curve25519 — the curve Algorand's VRF and your engine both use — is one specific well-studied choice of curve, designed to be fast and hard to implement insecurely.

> YOU WILL NOT NEED TO IMPLEMENT ANY OF THIS
>
> Every operation is provided by a library. What matters is knowing *what hardness assumption you are relying on*, because that is the sentence a reviewer will ask you for: **"if the discrete logarithm problem on Curve25519 were solved tomorrow, my scheme would break, and so would most of the internet."**

SECTION 6

Ed25519, and why determinism matters

Ed25519 is a specific signature scheme built on Curve25519. It is what your engine uses, and it has one property that turns out to be load-bearing.

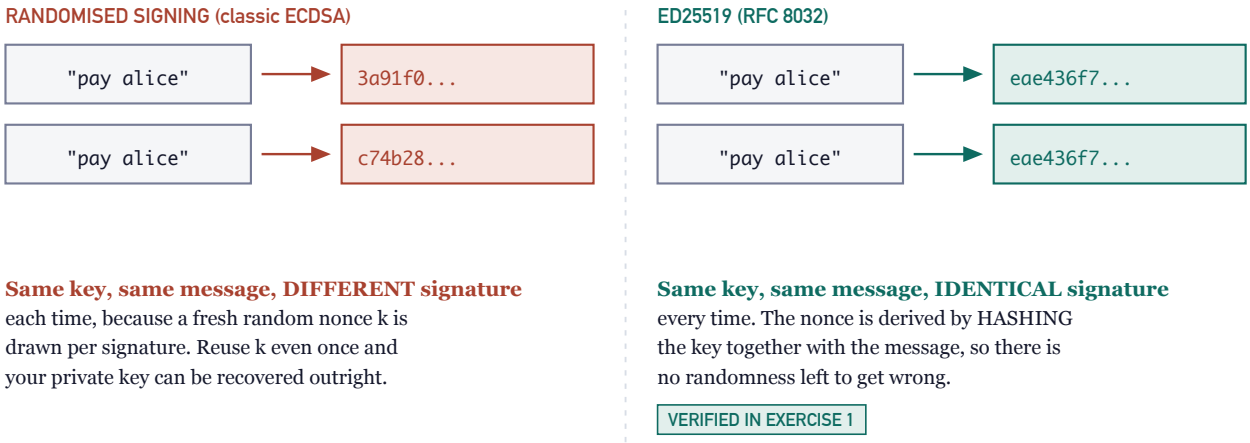


Figure 5 — Determinism is a safety feature in ECDSA. In Algorand it is load-bearing.

Most older signature schemes need a fresh random number — a **nonce** — for every signature. If that randomness is ever repeated or predictable, the private key can be recovered outright. This has caused real, expensive failures.

Ed25519 removes the risk by deriving the nonce deterministically: it hashes the private key together with the message. Same key, same message, same signature, forever.

WHY THIS MATTERS FAR MORE THAN USUAL FOR YOU

For most systems, Ed25519's determinism is just a safety feature — one less thing to get wrong.

For your project it is **the mechanism itself**. A VRF must produce exactly one output per (key, input), or a node could re-roll until it won the lottery. Ed25519's determinism is what your engine borrows to get that property for free.

That is the trick in `engine/vrf/vrf.go`, and section 11 is about exactly how far the trick can be trusted.

SECTION 7 • THE CENTRAL IDEA

Verifiable Random Functions

Everything so far has been preparation. This is the idea Algorand is built on, and the reason your project exists in the form it does.

The problem, stated precisely

We need to choose a random committee, weighted by stake, such that:

- nobody can **predict** who will be chosen, so nobody can be attacked in advance;
- nobody can **influence** who is chosen, including themselves;
- a chosen participant can **prove** they were chosen, so their vote counts;
- and there is **no organiser** — no trusted party running the draw, because a trusted party is exactly what we removed in Phase 1.

Each requirement alone is easy. Together they look contradictory: a random draw with no organiser that is nonetheless provable. The VRF resolves it.

What a VRF is

A function taking your private key and any public input, and returning two things:

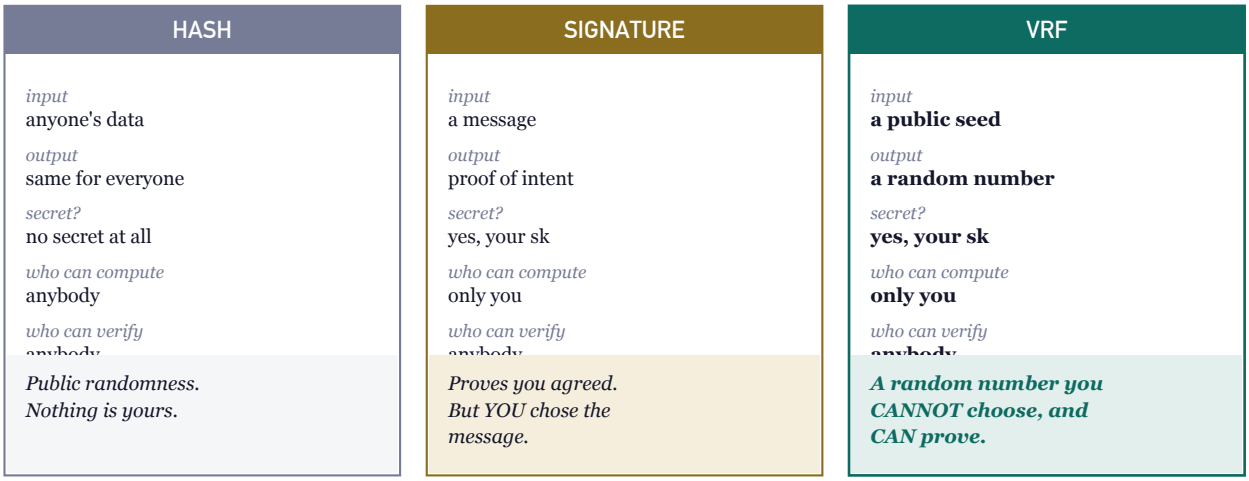
```
beta, pi  =  VRF(sk, alpha)

  alpha  the input        (public: everyone knows it)
  beta   the output       (looks completely random)
  pi     the proof        (lets anyone check beta, using only pk)
```

Its three properties

Uniqueness For a given *sk* and *alpha*, there is exactly **one** *beta* that will ever verify. You cannot re-roll, cannot search for a better answer, cannot produce two valid outputs. **This is the property that makes the lottery unriggable.**

Pseudorandomness	To anyone without *sk*, *beta* is indistinguishable from a random string. They cannot predict it, cannot narrow it down, cannot tell whether you won before you say so.
Provability	The proof *pi* convinces anyone holding your **public** key that *beta* really is the correct output for *alpha*. Your secret is never transmitted and never inferable.



THE ONE PROPERTY THAT MATTERS:

A signature lets you pick what you sign. A VRF does not — the output is fixed by your key and the **seed before you ever touch it. That is why it can run a lottery nobody can rig.**

Figure 6 — The VRF is the only one of the three that is private first and public afterwards.

THE SENTENCE WORTH MEMORISING

A signature proves **you agreed to a message you chose**.

A VRF gives you **a random number you could not choose, and can prove you did not choose**.

That difference is the whole thing. If you can say this cleanly, you understand Phase 2.

Why it feels impossible, and why it is not

The apparent paradox: how can a number be both unpredictable to others and fixed in advance for you?

Because it is determined by something only you hold. The output is a fixed function of `(sk, alpha)`, so it existed, mathematically, before you computed it — you are discovering it, not choosing it. And since nobody else has `sk`, nobody else can discover it early. The proof then lets them confirm, afterwards, that you reported honestly.

Ed25519's determinism (section 6) is exactly this shape, which is why it can be borrowed to build one.

SECTION 8

Sortition: running the secret lottery

"Sortition" means selection by lot. Algorand's version, from section 5 of the paper, is where the VRF earns its place.

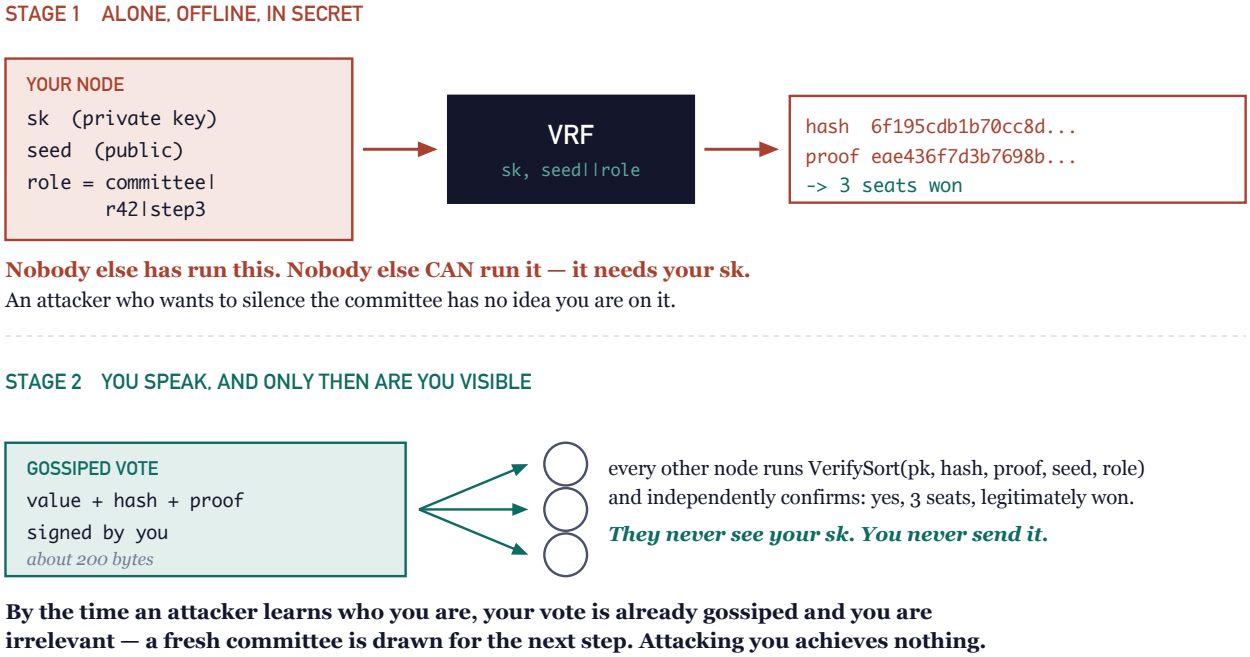


Figure 7 — The whole mechanism: you find out alone, everyone finds out afterwards.

The input is the whole design

```
beta, pi  =  VRF(sk, seed || role)

  seed  a public random value for this round, the same for everyone
  role  a label:  <"committee", round 42, step 3>
```

Change any part of `role` and you get a completely different, independent lottery. That is how Algorand draws a **fresh committee for every single step** at zero coordination cost — no messages, no organiser, no setup. Exercise 3 in section 14 demonstrates it in four lines.

PROPERTY NEEDED	WHICH VRF PROPERTY DELIVERS IT
Nobody can predict the committee	Pseudorandomness — without *sk*, *beta* is unguessable
Nobody can rig their own selection	Uniqueness — there is one answer and you cannot re-roll
A winner's vote is counted	Provability — *pi* convinces everyone, using only *pk*
No trusted organiser exists	Every node computes its own draw, alone

THE CONSEQUENCE THAT MAKES BA* POSSIBLE

Because selection is private until you speak, an attacker cannot silence the committee in advance — they do not know who is on it.

And because a fresh committee is drawn every step, by the time your message reveals you, **you no longer matter**. Attacking you afterwards accomplishes nothing.

This is what the Algorand paper calls resistance to an *adaptive adversary*, and it is the property that earlier systems like ByzCoin could not achieve. In Phase 4 you will meet it again as the reason committee members speak exactly once.

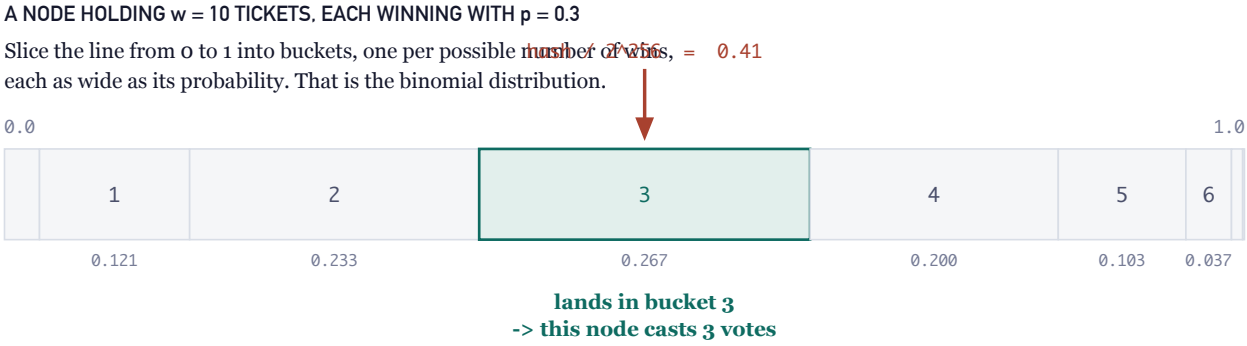
SECTION 9

From stake to votes: the binomial

A VRF gives you one random number. But influence must be proportional to stake — so how does one number become *many* votes?

The answer is to stop thinking of a node as a participant, and start thinking of each **coin** as one. A node with 10,000 coins holds 10,000 lottery tickets, each winning independently with probability `p = tau / W`, where `tau` is the target committee size and `W` the total stake.

The number of tickets that win therefore follows the **binomial distribution** — the same distribution as "how many heads in `w` tosses of a biased coin". Algorand calls each winning ticket a **sub-user**.



WHY BUCKETS AND NOT A SIMPLE COIN FLIP:

One vote per node would mean a node with 1000 coins has the same say as one with 1.
One draw per COIN makes influence exactly proportional to stake — and kills Sybils. See Figure 9.

Figure 8 — Your hash is a dart. Where it lands decides how many seats you won.

How the code does it

Your engine's `SubUsers()` walks the cumulative distribution:

```
p := tau / float64(W)
x := ratio(hash)              // the VRF hash, normalised to [0,1)
cum := 0.0
for j := uint64(0); j <= w; j++ {
    cum += math.Exp(logBinomPMF(j, w, p))
    if x < cum {
        return j              // this node won j seats
    }
}
```

TWO DETAILS IN THAT CODE WORTH UNDERSTANDING

Why logarithms. `logBinomPMF` works in log space via log-gamma. Computing `C(n,k)·p^k·q^(n-k)` directly overflows and loses all precision once *n* reaches the thousands — which is exactly the regime here, since *W* is the entire stake.

Why 53 bits. `ratio()` takes the top 53 bits of the hash, the mantissa width of a float64. Using more adds no precision, and the naive `float64(u)/2^64` can round up to exactly 1.0, which would fall off the end of the CDF walk and return the wrong seat count.

SECTION 10

Why the binomial kills Sybil attacks

Phase 1 said proof of stake defeats Sybils "arithmetically". This is the arithmetic.

ONE HONEST IDENTITY

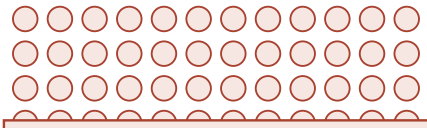


holds 10,000 coins
= 10,000 lottery tickets

`expected seats: 100.0`

`measured over 4,000 trials: 99.84`

100 SYBIL IDENTITIES



100 accounts,
100 coins each

`expected seats: 100.0`

`measured over 4,000 trials: 100.32`

The identity that makes this work:
`B(k1; n1, p) + B(k2; n2, p) = B(k1+k2; n1+n2, p)`
Two small binomials add up to one big one. The split is invisible to the maths.

Figure 9 — Splitting a stake across fake identities changes nothing. Verified in your own tools.

The binomial distribution has an addition property:

```
B(k1; n1, p) + B(k2; n2, p) = B(k1+k2; n1+n2, p)
```

In words: **two independent binomial draws with the same p add up to a single binomial draw over the combined size**. Ten thousand tickets held by one account, or split across a hundred accounts of a hundred tickets each, produce the same distribution of total wins.

So a Sybil attacker gains **nothing**. Not less than before — exactly the same. Splitting is invisible to the mathematics.

THIS IS CHECKPOINT QUESTION 2, AND IT IS WORTH REHEARSING

The naive design — one vote per node — would be catastrophic here: 100 fake accounts would give 100 votes instead of 1. It is precisely the per-coin draw that removes the incentive.

You can verify this yourself. `tools/verify_sortition.py` in your own project runs it as a Monte Carlo: one identity with 10,000 stake averaged **99.84** seats, a hundred Sybils with 100 each averaged **100.32**, against a theoretical 100.00.

Note carefully what this does **not** do: it does not make identities expensive, and it does not detect Sybils. It makes them *pointless*, which is stronger — there is nothing to detect and nothing to punish.

SECTION 11

What your engine actually ships

Time to be precise about the code you already have, including where it is weaker than the paper.

WHAT engine/vrf/vrf.go DOES



WHERE IT IS NOT A REAL VRF

A true VRF guarantees UNIQUENESS even when the key was chosen by an attacker:
for one (sk, input) there is exactly ONE output that will ever verify.

Ed25519 does not promise that. With non-canonical encodings or small-order points, a malicious key holder can, in some verifier implementations, produce more than one signature that verifies — and therefore more than one "random" draw.

WHY IT IS STILL ACCEPTABLE FOR YOUR PROJECT:

Fabric is permissioned — orderer keys are issued by a CA under an MSP, not chosen by an attacker.

The interface is swappable: drop in an RFC 9381 ECVRF and no other file changes.

Figure 10 — What your engine actually ships, and precisely where it falls short.

The construction

```
func (Ed25519VRF) Prove(sk, m) (hash, proof []byte) {
    proof = ed25519.Sign(sk, m)      // deterministic, per RFC 8032
    h := sha512.Sum512(proof)        // pseudorandom output
    return h[:], proof
}
```

Check it against the three properties. **Pseudorandomness**: the output is SHA-512 of a signature, so it is indistinguishable from random without `sk`. **Provability**: verification is signature verification

plus a hash comparison, needing only `pk`. **Uniqueness**: Ed25519 signing is deterministic, so for honest keys there is exactly one signature and therefore one output.

AND HERE IS THE GAP — STATE THIS PLAINLY IN YOUR WRITEUP

That uniqueness argument holds only for **honestly generated keys**. A true VRF guarantees uniqueness even when the key was chosen adversarially.

Ed25519 does not promise that. Using non-canonical scalar encodings or small-order curve points, a malicious key holder can — in some verifier implementations — produce more than one signature that verifies, and therefore more than one "random" draw. They could pick whichever wins them more seats.

Why it is nonetheless the right call here: Fabric is permissioned. Orderer keys are issued by a certificate authority under an MSP, not generated by an attacker. The threat the gap opens is one your deployment model already closes.

And `vrf.go` defines a `VRF` interface, so a standards-track implementation drops in without touching another file.

SECTION 12

Real VRFs, and when you would need one

The standards-track construction is **ECVRF**, specified in RFC 9381 (2023). Algorand's paper uses an earlier elliptic-curve VRF of the same family.

You do not need to implement it, and for this project you almost certainly should not. But you should be able to say what it adds:

	YOUR Ed25519 SHORTCUT	ECVRF (RFC 9381)
Pseudorandomness	Yes	Yes
Provability	Yes	Yes
Uniqueness, honest keys	Yes	Yes
Uniqueness, adversarial keys	**No**	Yes — proven
Lines of code	About 15	Several hundred, easy to get subtly wrong
Right for a permissioned lab network	Yes	Overkill
Right for an open network	No	Yes

ECVRF works by hashing the input *onto the curve* to get a point, multiplying that point by the secret scalar, and attaching a zero-knowledge proof that the same scalar was used as in the public key. The uniqueness is then a theorem about the curve group, not a property of one signing algorithm's conventions.

THE HONEST POSITION FOR YOUR REPORT

"We implement the VRF using Ed25519's deterministic signatures, which satisfies pseudorandomness and provability, and satisfies uniqueness for honestly generated keys. Since Hyperledger Fabric is permissioned and orderer keys are issued by a CA, adversarial key generation is outside our threat model. The VRF is defined behind an interface so an RFC 9381 ECVRF can replace it without changes elsewhere."

That is a stronger answer than pretending the gap is not there. Knowing exactly where your implementation departs from the paper — and why the departure is safe in your setting — is the thing that distinguishes a considered project from a copied one.

SECTION 13

Glossary

New terms from this phase. Phase 1's glossary still applies underneath.

alpha	Conventional name for a VRF's input. Public. In Algorand it is `seed || role`. *Section 7*
beta	Conventional name for a VRF's output — the pseudorandom value. *Section 7*
Binomial distribution	The distribution of "how many successes in n independent trials". Governs how many committee seats a stake wins. *Section 9*
Birthday bound	A collision in an n-bit hash appears after about 2^(n/2) tries, not 2^n, because every new item is compared against all previous ones. *Section 3*
Collision resistance	Cannot find **any** two inputs with the same hash. The weakest of the three hash properties, because the attacker chooses both inputs. *Section 2*
Curve25519	The specific elliptic curve used by Ed25519, Algorand's VRF, and your engine. Chosen for speed and for being hard to implement insecurely. *Section 5*
Discrete logarithm	Recovering k from the point k·G. Believed infeasible; this is the hardness every elliptic-curve key pair rests on. *Section 5*
ECVRF	The standards-track VRF construction, RFC 9381. Provides uniqueness even for adversarially generated keys. *Section 12*
Ed25519	A signature scheme on Curve25519 whose signing is fully deterministic. *Section 6*
Generator (G)	A fixed, publicly agreed starting point on the curve. *Section 5*
Inclusion proof	The sibling hashes needed to show a leaf belongs to a Merkle root, without sending the other leaves. *Section 4*
Merkle tree	A tree of hashes committing to a set of items with a single root value. *Section 4*
Nonce	A number used once. In signatures, the per-signature random value that Ed25519 replaces with a deterministic hash. *Section 6*
Preimage resistance	Given an output, cannot find any input producing it. *Section 2*

Proof (pi)	The VRF's third output: convinces a verifier holding pk that beta is correct for alpha. *Section 7*
Pseudorandomness	Indistinguishable from random to anyone lacking the secret key. One of the three VRF properties. *Section 7*
Role	The label distinguishing one sortition lottery from another — name, round, step. Changing it yields an independent draw. *Section 8*
Second-preimage resistance	Given one input, cannot find a different one with the same hash. What the block chain's immutability rests on. *Section 2*
Seed	The public per-round randomness fed into sortition, identical for all nodes. *Section 8*
Sortition	Selection by lot. Algorand's private, verifiable, stake-weighted committee draw. *Section 8*
Sub-user	One winning lottery ticket. A node holding many coins may win many, and casts that many votes. *Section 9*
tau	The target committee size — the expected total number of seats handed out. *Section 9*
Uniqueness	For one (sk, alpha) there is exactly one beta that verifies. The property that makes the lottery unriggable — and the one the Ed25519 shortcut only partly guarantees. *Sections 7, 11*
VRF	Verifiable Random Function. Produces a random value only you can compute and anyone can verify. The idea Algorand is built on. *Section 7*
W	Total stake across all participants. Sets p = tau/W. *Section 9*

SECTION 14 • HANDS-ON

Four exercises, all runnable now

These need only the `cryptography` package, which is already installed on your machine. Every output below is real — each was executed while this document was written.

Exercise 1 — prove Ed25519 is deterministic

```
from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey
import hashlib

sk  = Ed25519PrivateKey.from_private_bytes(bytes(range(32)))
pk  = sk.public_key()
msg = b'pay alice 500'

s1, s2 = sk.sign(msg), sk.sign(msg)
print('signature #1 :', s1.hex()[:32], '...')
print('signature #2 :', s2.hex()[:32], '...')
print('identical?   :', s1 == s2)
```

OUTPUT

```
signature #1 : eae436f7d3b7698b1ea031c37ef813aa ...
signature #2 : eae436f7d3b7698b1ea031c37ef813aa ...
identical?   : True
```

Run it twice more. The bytes never change. That single fact is what section 6 said is load-bearing: it means one (key, message) has exactly one signature, so a node cannot re-roll its lottery.

Exercise 2 — build a VRF from that determinism

```
from cryptography.exceptions import InvalidSignature

def vrf_prove(sk, alpha):
    proof = sk.sign(alpha)                      # deterministic
    return hashlib.sha512(proof).digest(), proof

def vrf_verify(pk, alpha, beta, proof):
    try:
        pk.verify(proof, alpha)
    except InvalidSignature:
        return False
    return hashlib.sha512(proof).digest() == beta

alpha = b'seed-round-42|committee|step-3'
beta, proof = vrf_prove(sk, alpha)
print('beta        :', beta.hex()[:32], '...')
print('verify      :', vrf_verify(pk, alpha, beta, proof))
print('tampered    :', vrf_verify(pk, alpha, b'\x00'*64, proof))
print('wrong input :', vrf_verify(pk, b'different', beta, proof))
```

OUTPUT

```
beta        : 0ce7266e59515a5d3fb8b43055c9ea93 ...
verify      : True
tampered    : False
wrong input : False
```

You have just written the same construction as `engine/vrf/vrf.go`. Note what verification needs: only the public key. The secret never travels.

Exercise 3 — one seed, many independent lotteries

```
for role in [b'committee|r42|s1', b'committee|r42|s2', b'proposer|r42']:
    b, _ = vrf_prove(sk, role)
    print(f'{role.decode():22} -> {b.hex()[:24]}...')
```

OUTPUT

```
committee|r42|s1       -> 043d10fbc4c55a444b640cd4...
committee|r42|s2       -> 599a7e1df67d7c723b886cba...
proposer|r42           -> 6f195cdb1b70cc8d08d34265...
```

Three completely unrelated outputs from one key, with no coordination and no extra messages. This is how Algorand affords a fresh committee for every step.

Exercise 4 — measure the birthday bound yourself

```
import itertools

def find_collision(nbits):
    seen = {}
    for n, i in enumerate(itertools.count()):
        d = hashlib.sha256(str(i).encode()).digest()
        t = int.from_bytes(d, 'big') >> (256 - nbits)   # truncate
        if t in seen:
            return n
        seen[t] = i

for bits in (16, 24, 32, 40):
    tries  = find_collision(bits)
    theory = int(2 ** (bits / 2))
    print(f'{bits:>3} bits: {tries:>10,} tries   (2^(n/2) = {theory:>9,})')
```

OUTPUT

```
 16 bits:        251 tries   (2^(n/2) =       256)
 24 bits:      8,864 tries   (2^(n/2) =     4,096)
 32 bits:     95,303 tries   (2^(n/2) =    65,536)
 40 bits:  1,526,146 tries   (2^(n/2) = 1,048,576)
```

The measured counts hug 2^(n/2), not 2^n. This is the experiment that produced Figure 2 — you are reproducing the document's own data.

Exercise 5 — the one that matters most

Open your own project and run the sortition validator that already exists:

```
cd algorand-fabric/tools
python3 verify_sortition.py
```

It runs five Monte Carlo checks on the exact algorithm in `engine/vrf/sortition.go`. Read the output against section 9 and section 10 and confirm you can explain each result — particularly test 2, where one identity holding 10,000 stake and a hundred Sybils holding 100 each both average almost exactly 100 seats.

> **THEN READ THE GO**
>
> With sections 7 to 11 fresh, open `engine/vrf/sortition.go` and read it top to bottom. It is 170 lines and you should now recognise every one: the role encoding, the VRF call, the binomial CDF walk, the log-space arithmetic, the 53-bit ratio.
>
> If any line is still opaque, the section that explains it is named in the comment above it. That file was written to be read alongside this document.

SECTION 15

Checkpoint

Answer these out loud, without notes, before Phase 3. Questions 1, 4 and 9 are the ones you are most likely to be asked about this project.

01	Name the three properties of a VRF, and say which one makes the lottery impossible to rig.	§7
02	A signature and a VRF both need your private key and both are publicly verifiable. State the difference in one sentence.	§7
03	Why can a node not simply keep re-computing its VRF until it wins a seat?	§7, 6
04	Explain why splitting 10,000 coins across 100 accounts wins an attacker no extra committee seats. Use the binomial identity.	§9, 10
05	Sortition's input is `seed || role`. What breaks if you drop the role and use the seed alone?	§8
06	Why does SHA-256 give 128-bit collision resistance rather than 256-bit?	§3
07	Which hash property does a blockchain's immutability actually depend on — preimage, second-preimage, or collision resistance? Justify it.	§2, 6 of Phase 1
08	What is the discrete logarithm problem, and what in your project would break if it were solved?	§5
09	Your engine builds its VRF from an Ed25519 signature. State precisely which VRF property is only partly guaranteed, and why that is acceptable for Fabric.	§11
10	An attacker learns you are on the committee the moment you broadcast your vote. Why does attacking you at that point gain them nothing?	§8

SECTION 15 • MODEL ANSWERS

Answers

Same ground in your own words counts. The mechanism matters, the wording does not.

01 **Uniqueness**, **pseudorandomness**, and **provability**. Uniqueness makes it unriggable: for a given key and input there is exactly one output that will ever verify, so there is no re-rolling and no searching for a better draw.

02 A signature proves you agreed to a message **you chose**; a VRF gives you a random number you **could not choose**. The output is fixed by your key and the public input before you ever compute it.

03 Because the output is deterministic. There is only one valid answer for a given (sk, input), so recomputing produces the identical result every time. Re-rolling would require a different input — and the input is fixed by the public seed and the role, both of which everyone else can check.

04 Because the binomial distribution adds: `B(k1;n1,p) + B(k2;n2,p) = B(k1+k2;n1+n2,p)`. Each coin is an independent ticket with the same win probability p, so a hundred draws over 100 tickets have exactly the same combined distribution as one draw over 10,000. The split is invisible to the maths — measured at 100.32 seats versus 99.84 unsplit, against a theoretical 100.

05 Every step of every round would use the same lottery, so the same nodes would be selected every time. The committee would stop being fresh, which destroys the two properties that matter: an attacker could learn the membership once and target it thereafter, and the replacement of members after each step — which is what makes attacking a revealed member pointless — would no longer happen.

06 The birthday bound. An attacker seeking a collision gets to choose both inputs, and every new candidate is compared against all previously seen ones, so matching pairs accumulate quadratically. A collision therefore appears after roughly `2^(n/2)` attempts rather than `2^n`.

07 **Second-preimage resistance.** The chain's guarantee is that you cannot swap the contents of an existing block while keeping its hash — that is, given a specific existing input, you cannot find a different one with the same hash. Collision resistance is a separate and stronger requirement, needed to stop an attacker crafting two colliding blocks from scratch before either is committed.

08 Recovering the scalar k from the curve point k·G. It is believed infeasible, and it is what makes a private key unrecoverable from a public key. If it were solved, anyone could derive any node's private key from its public key — forging signatures, forging VRF proofs, and letting an attacker claim any number of committee seats. The entire project would collapse, along with most of the internet's transport security.

09 **Uniqueness**, and only for adversarially generated keys. Ed25519 signing is deterministic, so an honest key yields exactly one signature and therefore one output — but Ed25519 does not guarantee signature uniqueness when the key itself was chosen maliciously (non-canonical encodings, small-order points). It is acceptable because Fabric is permissioned: orderer keys are issued by a CA under an MSP, so adversarial key generation is outside the threat model. The VRF sits behind an interface so a real ECVRF can replace it.

10 Because committee membership is per-step, and a fresh committee is drawn for the next step from an independent lottery. Your vote is already gossiped by the time you are visible, and you have no special role in any future step — so silencing you removes nothing. This is what makes Algorand resistant to an adaptive adversary, and it is why members speak exactly once.

PHASE 2 DONE — WHAT CHANGES NOW

You can read `engine/vrf/` in full. That is a third of your project's consensus engine no longer a black box.

Next is **Phase 3 — Distributed systems and consensus theory**, sixteen hours and the hardest of the eight. It is also the one the roadmap says never to cut: it is where the 2/3 bound stops being a rule you were told and becomes something you can derive.

If you would rather build for a while, Phase 5 (Go) needs only the Homebrew and Go installs, which are still outstanding.